

Algorithms from Revision (From Page 107)

Explanation + Steps + Pseudocode

Algorithm 1: Divide and Conquer – Sum of N Numbers

Problem Description: Given a list of N numbers, compute their sum using the divide-and-conquer technique.

Algorithm Explanation: The problem is divided into two subproblems of approximately equal size. Each subproblem computes the sum of its part. The partial sums are then combined to produce the final result. This approach enables parallel execution of subproblems.

Steps:

1. If the list has only one element, return it.
2. Divide the list into two halves.
3. Recursively compute the sum of each half.
4. Add the two partial sums.
5. Return the final sum.

Pseudocode:

```
function SUM(A, n)
  if n == 1 then
    return A[0]
  left_sum = SUM(A, n/2)
  right_sum = SUM(A + n/2, n - n/2)
  return left_sum + right_sum
```

Algorithm 2: Divide and Conquer – Finding Minimum Number

Problem Description: Given a list of numbers, find the minimum value using a recursive divide-and-conquer approach.

Algorithm Explanation: The list is divided into two sublists. The minimum of each sublist is computed recursively. The smaller of the two minimum values is returned as the final result.

Steps:

1. If the list contains one element, return it.
2. Divide the list into two parts.
3. Recursively find the minimum of the left part.
4. Recursively find the minimum of the right part.
5. Compare both results and return the smaller one.

Pseudocode:

```
function MIN(A, n)
  if n == 1 then
    return A[0]
  lmin = MIN(A, n/2)
  rmin = MIN(A + n/2, n - n/2)
  if lmin < rmin then
    return lmin
  else
    return rmin
```

Algorithm 3: Parallel Quick Sort

Problem Description: Sort a list of elements using the quicksort algorithm while exploiting parallelism.

Algorithm Explanation: Quicksort selects a pivot and partitions the list into elements smaller and larger than the pivot. The two partitions are then sorted recursively and can be executed in parallel.

Steps:

1. Select a pivot element.
2. Partition the list into two sublists (less than pivot, greater than pivot).
3. Recursively apply quicksort on both sublists in parallel.
4. Combine the sorted sublists.

Pseudocode:

```
procedure QUICKSORT(A, low, high)
  if low < high then
    pivot = PARTITION(A, low, high)
    QUICKSORT(A, low, pivot - 1)
    QUICKSORT(A, pivot + 1, high)
```

Algorithm 4: Closest Pair of Points

Problem Description: Given a set of points in a 2D plane, find the pair of points with the minimum distance between them.

Algorithm Explanation: The points are first sorted by x-coordinate. The set is divided into two halves, and the closest pair is found recursively in each half. A strip around the dividing line is then checked for a closer pair.

Steps:

1. Sort all points by x-coordinate.
2. Divide the points into left and right subsets.
3. Recursively find the closest pair in each subset.
4. Compute the minimum distance d from both halves.
5. Build a strip of points within distance d of the dividing line.
6. Find the closest pair in the strip.
7. Return the overall minimum distance.

Pseudocode:

```
function CLOSEST_PAIR(P)
  sort P by x-coordinate
  divide P into left and right
  left_min = CLOSEST_PAIR(left)
  right_min = CLOSEST_PAIR(right)
  d = min(left_min, right_min)
  build strip of points within d
  find minimum distance in strip
  return d
```